

Apply Filters to SQL Queries

Investigating login activity and employee machines in the organization database

Project description

As a security professional at a large organization, part of my role is to investigate potential security issues to help keep the system secure. In this project, I used SQL filters to examine the organization's `log_in_attempts` and `employees` tables, retrieving specific records to investigate suspicious login activity and to identify which employee machines needed security updates. I applied the AND, OR, and NOT operators, the LIKE keyword, and date and time filters to return only the records relevant to each investigation.

Retrieve after hours failed login attempts

To investigate a potential security incident that occurred after business hours, I queried the `log_in_attempts` table for all failed login attempts that took place after 18:00. The `login_time` column stores the time of each attempt, and the `success` column indicates the outcome, where a value of 0 (equivalent to FALSE) means the attempt failed. I used the AND operator so that both conditions must be true at once: the attempt happened after 18:00 and it was unsuccessful.

```
SELECT * FROM log_in_attempts
WHERE login_time > '18:00' AND success = 0;
```

This returns every row where the login occurred after 6:00 p.m. and failed, which are the after-hours failed attempts worth investigating.

```
MariaDB [organization]> SELECT *
-> FROM log_in_attempts
-> WHERE login_time > '18:00' AND success = 0;
```

Screenshot: after-hours failed login attempts query in the MariaDB shell.

Retrieve login attempts on specific dates

A suspicious event occurred on 2022-05-09, so I reviewed all login attempts from that day and the day before. The `login_date` column stores the date of each attempt. I used the OR operator so that a row is returned if it matches either date.

```
SELECT * FROM log_in_attempts
WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

This returns all login attempts recorded on either 2022-05-09 or 2022-05-08.

```
MariaDB [organization]> SELECT * FROM log_in_attempts
-> WHERE login_date = '2022-05-09' OR login_date = '2022-05-08';
```

Screenshot: login attempts on 2022-05-09 or 2022-05-08 query in the MariaDB shell.

Retrieve login attempts outside of Mexico

To investigate login attempts that did not originate in Mexico, I queried the `log_in_attempts` table and excluded any attempt from Mexico. The `country` column refers to Mexico using both `MEX` and `MEXICO`, so I used the `LIKE` keyword with the pattern `'MEX%'` to match both values. The percent sign (%) is a wildcard that matches any characters following `MEX`. I then used `NOT` to return only the rows that do not match that pattern.

```
SELECT * FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

This returns all login attempts that occurred outside of Mexico.

```
MariaDB [organization]> SELECT * FROM log_in_attempts
-> WHERE NOT country LIKE 'MEX%';
```

Screenshot: login attempts outside of Mexico query in the MariaDB shell.

Retrieve employees in Marketing

My team needed to perform security updates on the machines of employees in the Marketing department located in the East building. The `department` column stores the department, and the `office` column stores the office, with values such as `East-170` and `East-320` for the East building. I used the `AND` operator so that both conditions must be true, and the `LIKE` keyword with `'East%'` to match every office in the East building.

```
SELECT * FROM employees
WHERE department = 'Marketing' AND office LIKE 'East%';
```

This returns all Marketing employees whose office is located in the East building.

```
MariaDB [organization]> SELECT * FROM employees
-> WHERE department = 'Marketing'
-> AND office LIKE 'East%';
```

Screenshot: Marketing employees in the East building query in the MariaDB shell.

Retrieve employees in Finance or Sales

To perform a security update on machines belonging to employees in the Sales and Finance departments, I queried the `employees` table for anyone in either department. The `department` column stores the department, and I used the `OR` operator so that a row is returned if the employee is in Finance or in Sales.

```
SELECT * FROM employees
WHERE department = 'Finance' OR department = 'Sales';
```

This returns all employees in either the Finance or the Sales department.

```
MariaDB [organization]> SELECT * FROM employees  
-> WHERE department = 'Finance'  
-> OR department = 'Sales';
```

Screenshot: Finance or Sales employees query in the MariaDB shell.

Retrieve all employees not in IT

The Information Technology department had already received an update that all other departments still needed, so I queried the employees table for everyone not in IT. The department column stores the department, and I used the NOT operator to exclude employees in Information Technology.

```
SELECT * FROM employees  
WHERE NOT department = 'Information Technology';
```

This returns all employees who are not in the Information Technology department.

```
MariaDB [organization]> SELECT * FROM employees  
-> WHERE NOT department = 'Information Technology';
```

Screenshot: employees not in the Information Technology department query in the MariaDB shell.

Summary

I used SQL filters to investigate login activity and identify employee machines in the organization's log_in_attempts and employees tables. I applied AND and OR to filter on multiple conditions, NOT to exclude records, the LIKE keyword with the % wildcard to match patterns such as country and office values, and comparison operators to filter by date and time. Together, these queries returned the specific records needed to investigate the potential security issues and to target the correct machines for updates.